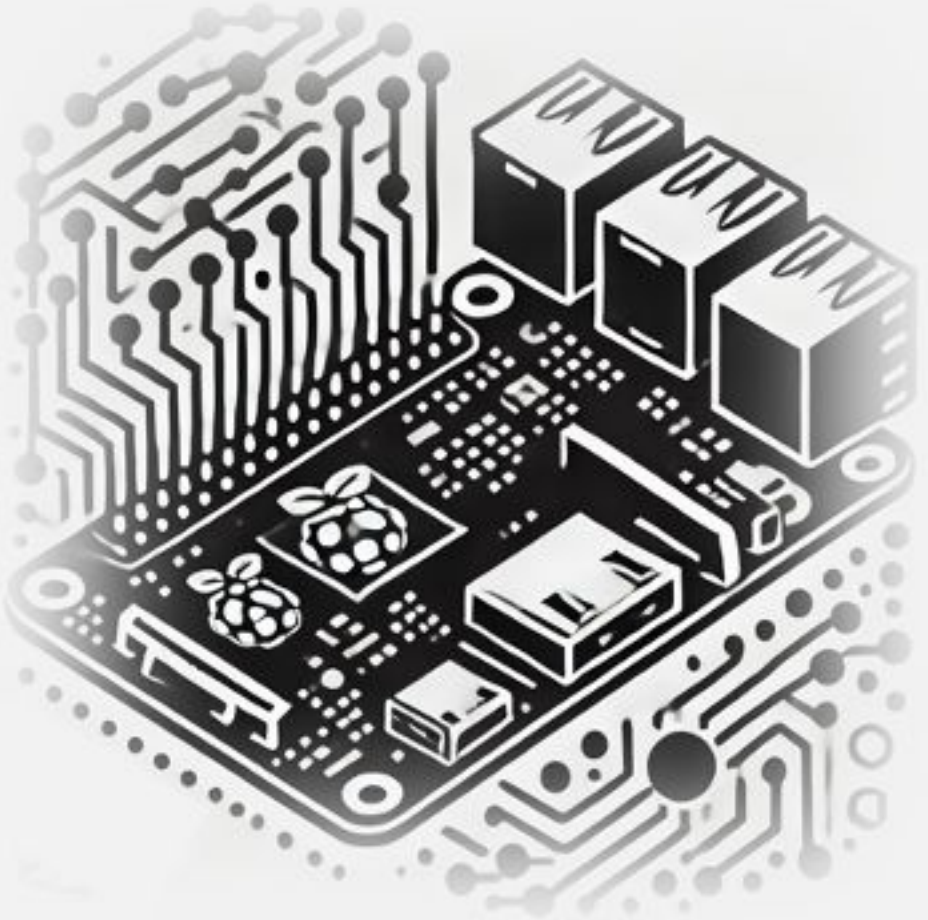
IESTI05 – Edge AI

Machine Learning System Engineering



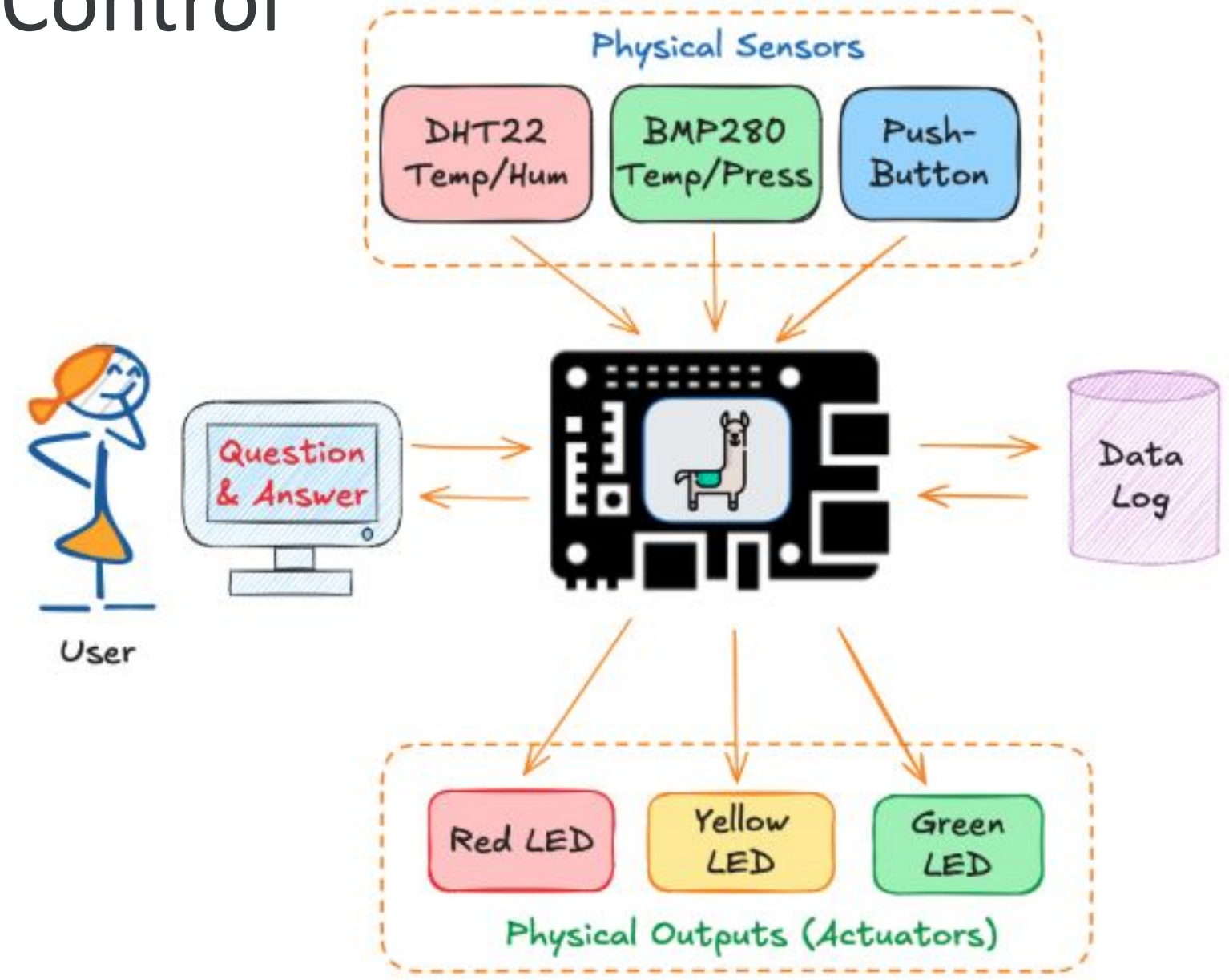
22. SLM for IoT Control

UNIFEI

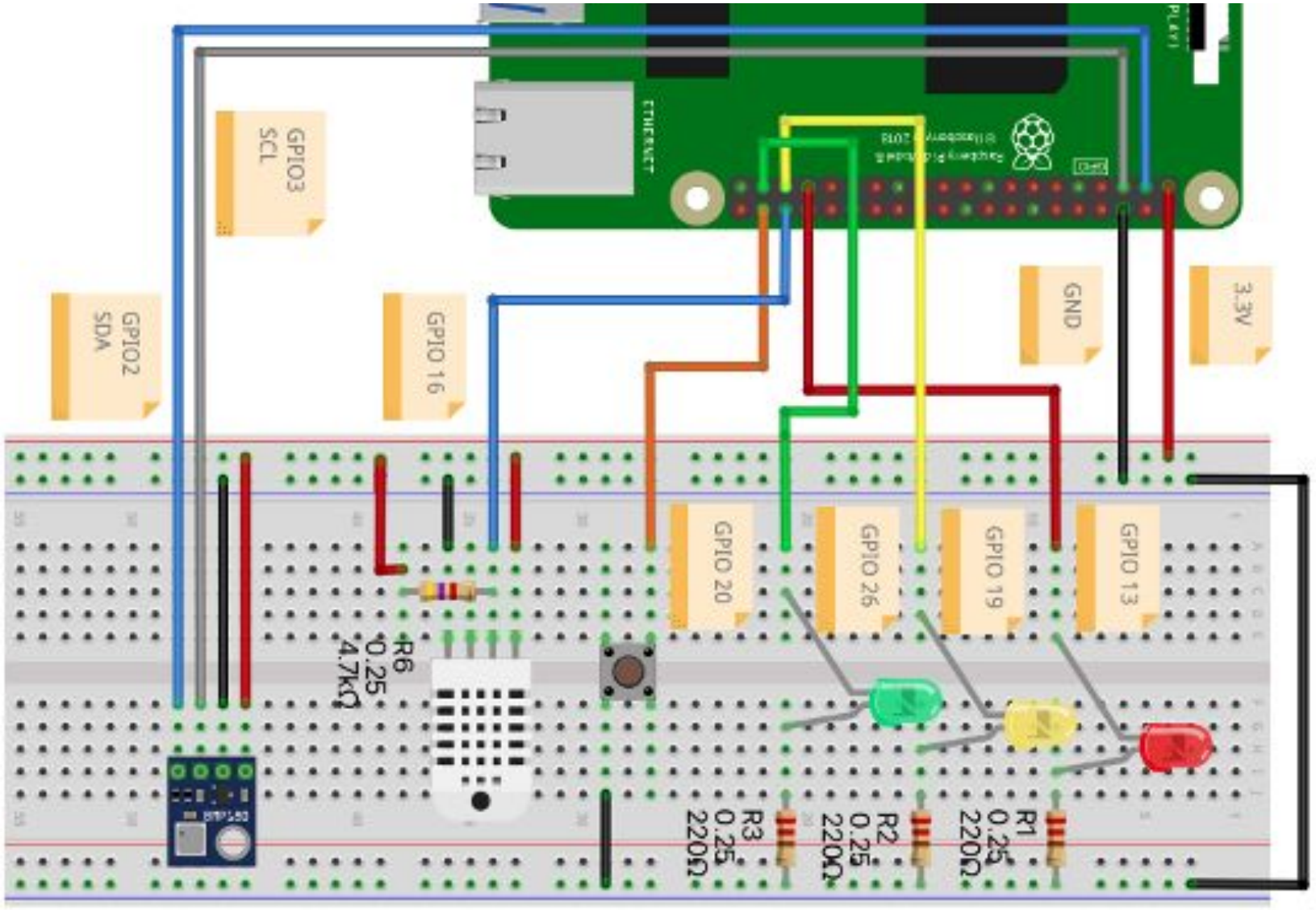
Marcelo Rovai

Professor Honoris Causa

SLMs for IoT Control



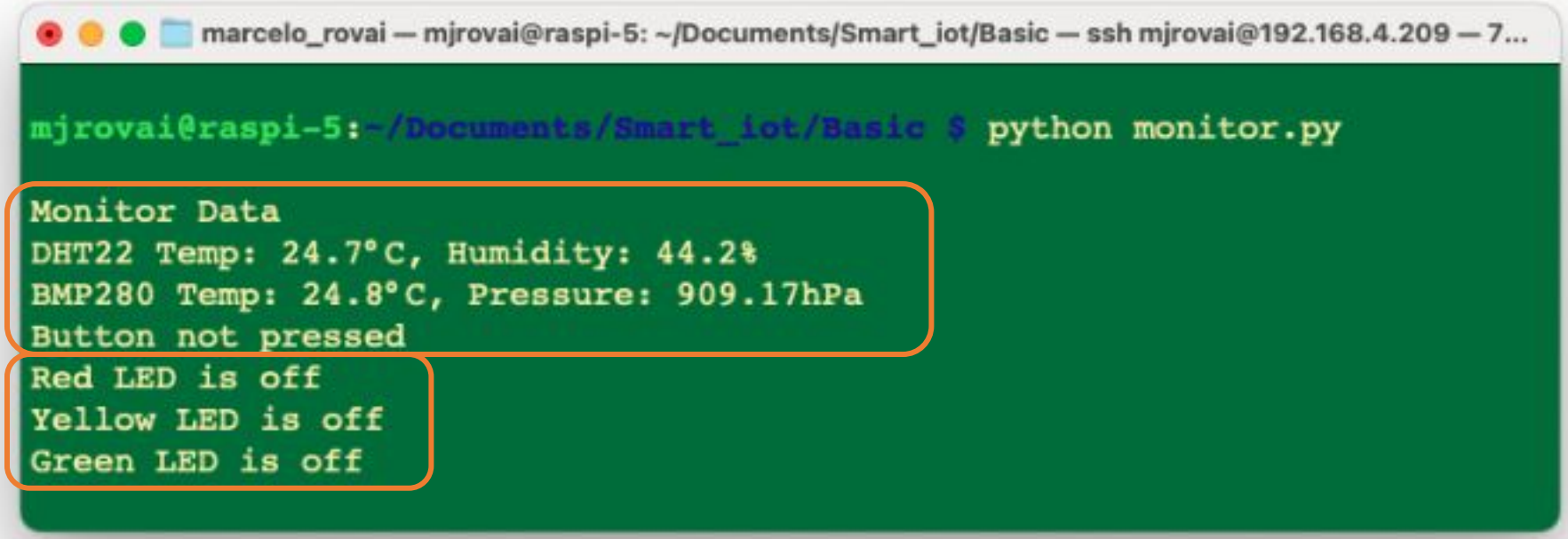
Hardware


GPIO3
SCL
GPIO2
SDA
GPIO 16
GND
3.3V
GPIO 20
GPIO 26
GPIO 19
GPIO 13
R6
0.25
4.7kΩ
R3
0.25
220Ω
R2
0.25
220Ω
R1
0.25
220Ω

Monitor.py

- GPIO Initialization

- collect_data()
- led_status()

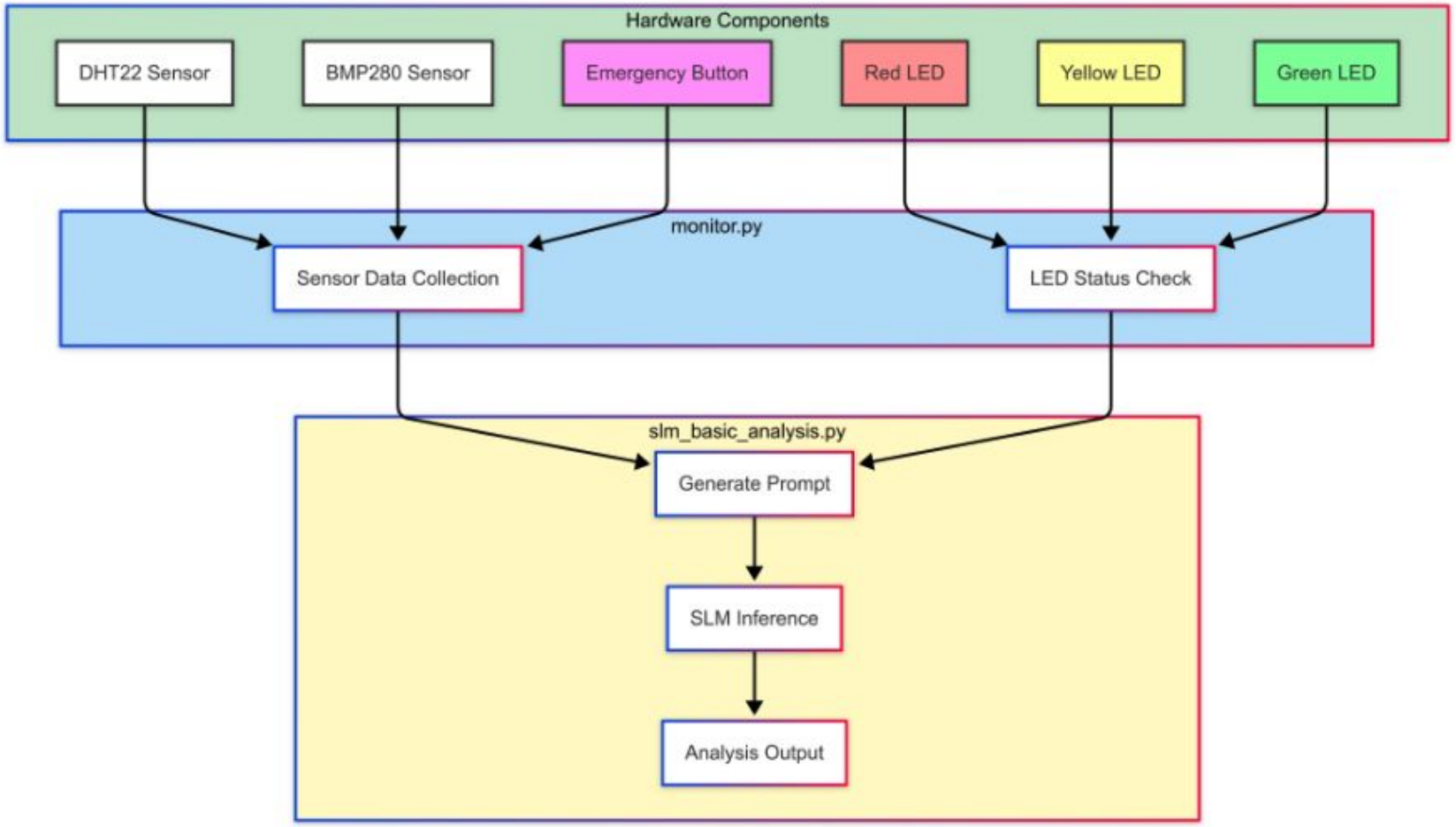


- control_leds(red, yellow, green)

monitor.py

SLM Basic Analysis

- The button, not pressed, shows a **normal operation**
- The button, when pressed, shows an **emergency**
- If the temperature is over 20°C, it means a **warning situation**
- The red LED, when on, indicates a problem/emergency.
- The yellow LED indicates a warning situation when on.
- The green LED is on, indicating the system is OK.

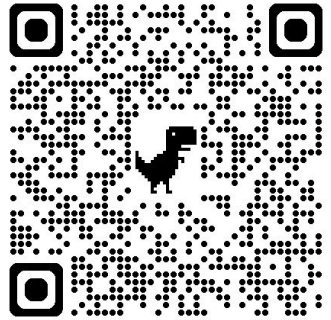


slm_basic_analysis.py

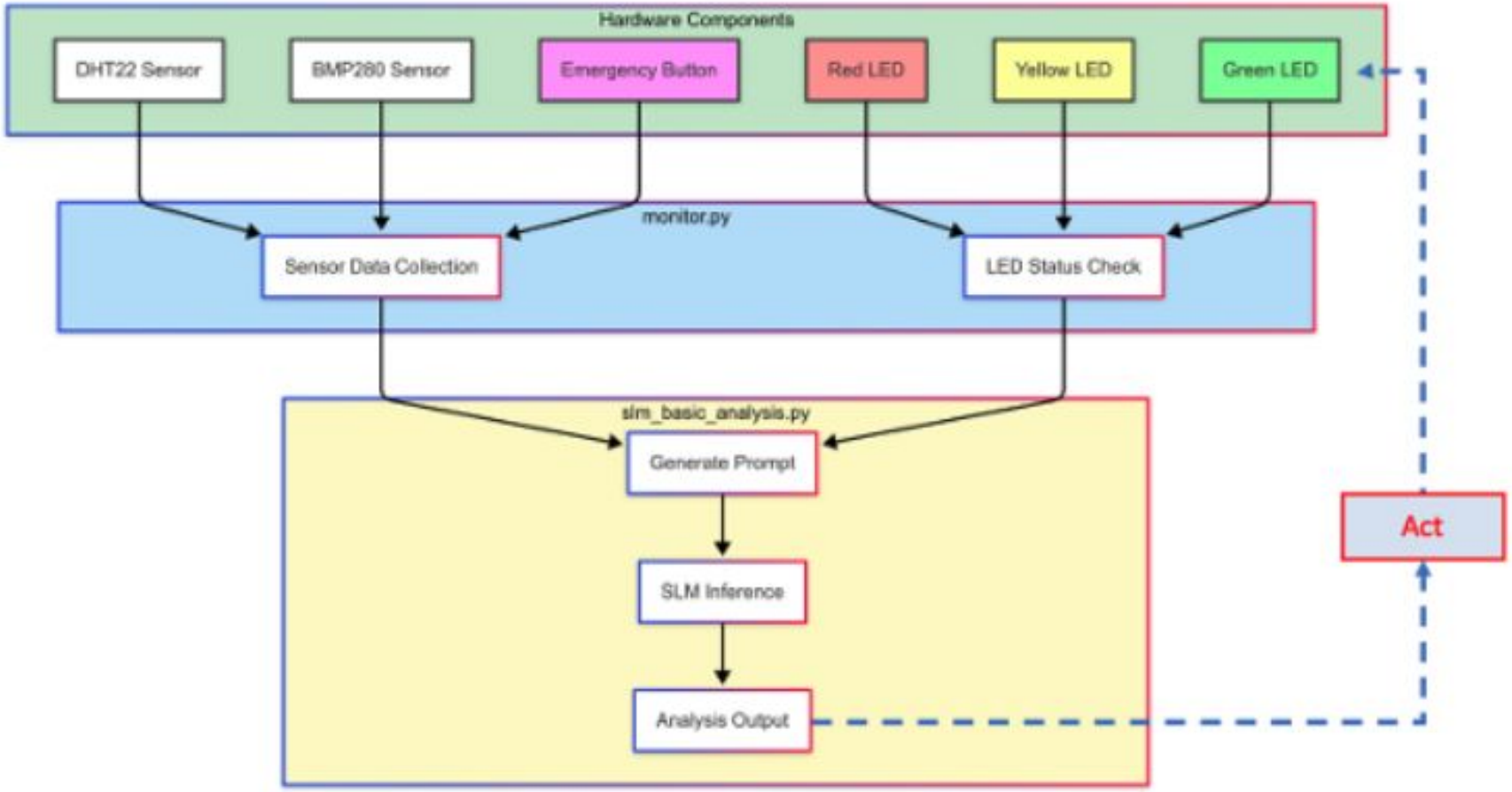
SLMs and IoT Control

Projects

SLM_IoT.ipynb



Acting on Output (Actuators)



slm_basic_act_leds.py

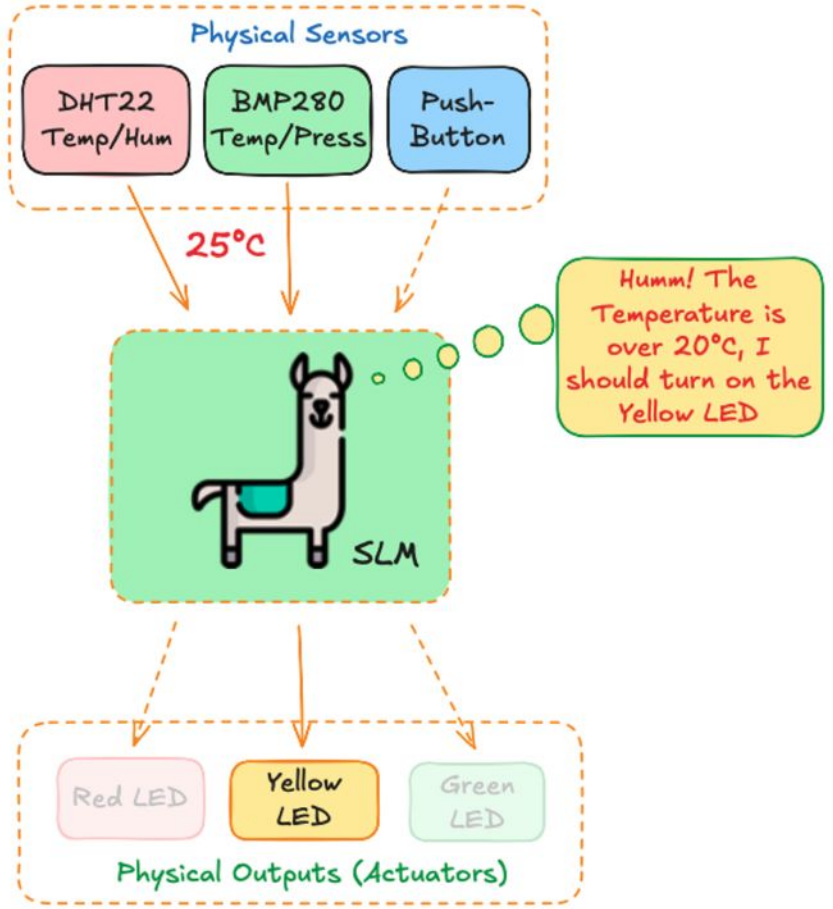
Prompting Engineering

Key Changes in the code:

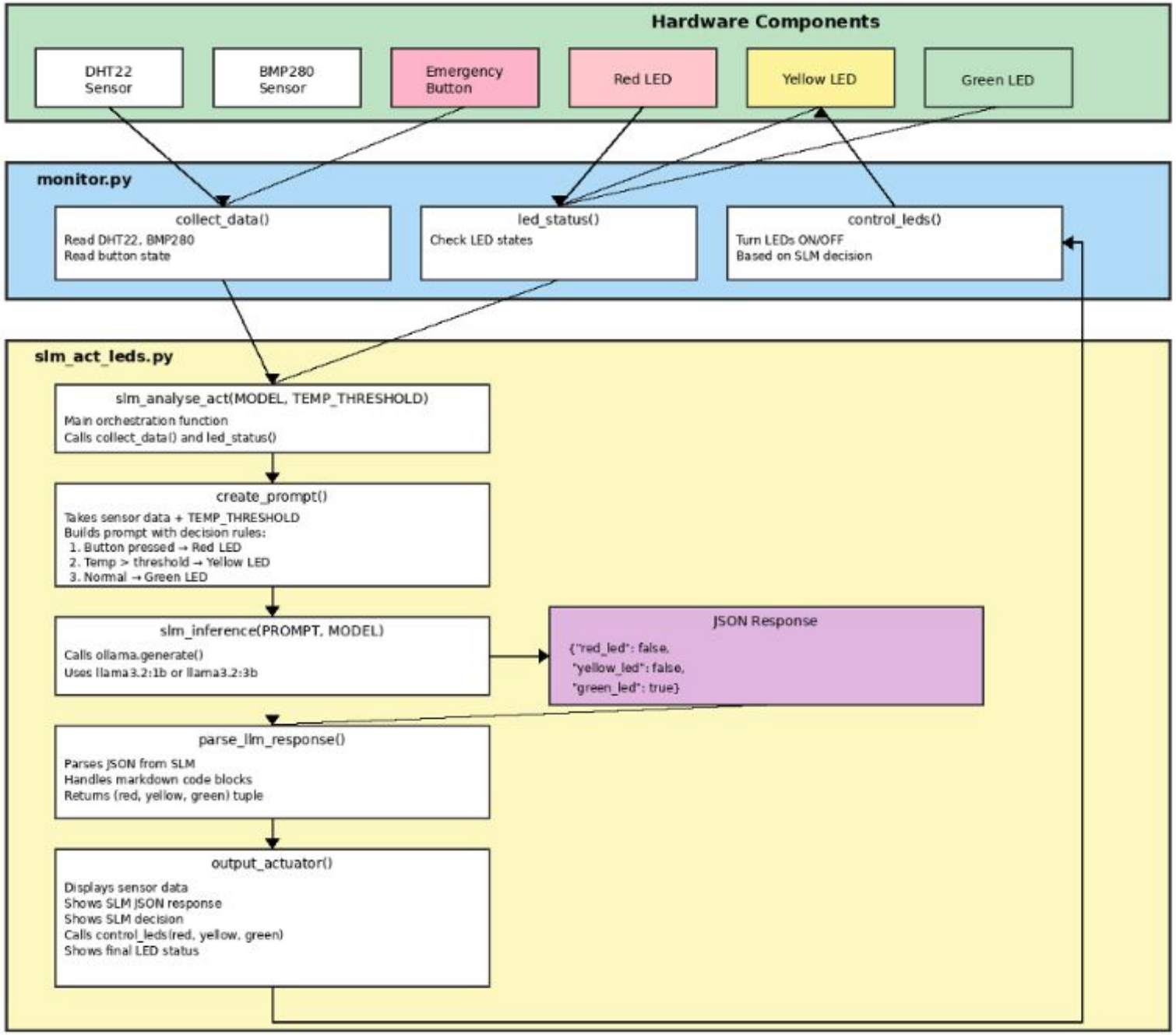
- Added JSON import
- Changed to JSON format - The prompt now asks for a structured JSON response:
- {"red_led": true, "yellow_led": false, "green_led": false}
- Updated parser - Now parses JSON instead of searching for text strings. Includes error handling and fallback to safe state (all LEDs off) if parsing fails.

Why JSON is Better:

- More reliable: No ambiguity about which LEDs to activate
- Structured: Clear true/false values instead of parsing text
- Error-resistant: The parser handles markdown code blocks (some models wrap JSON in ```) and provides safe fallback
- Flexible: Easy to add more fields later if needed

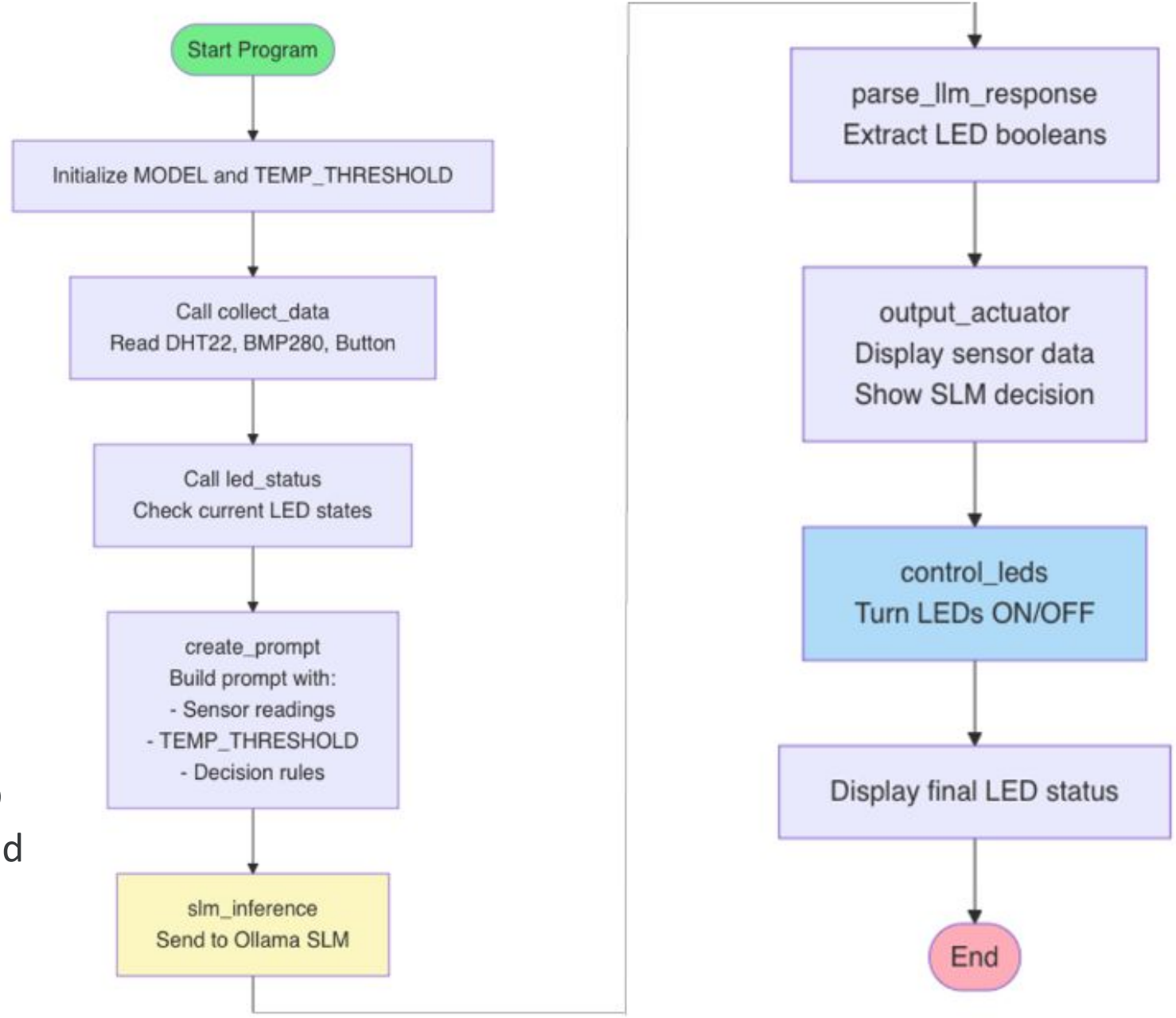


slm_act_leds.py


Hardware Components
DHT22 Sensor
BMP280 Sensor
Emergency Button
Red LED
Yellow LED
Green LED
monitor.py
collect_data()
Read DHT22, BMP280
Read button state
led_status()
Check LED states
control_leds()
Turn LEDs ON/OFF
Based on SLM decision
slm_act_leds.py
slm_analyse_act(MODEL, TEMP_THRESHOLD)
Main orchestration function
Calls collect_data() and led_status()
create_prompt()
Takes sensor data + TEMP_THRESHOLD
Builds prompt with decision rules:
1. Button pressed → Red LED
2. Temp > threshold → Yellow LED
3. Normal → Green LED
slm_inference(PROMPT, MODEL)
Calls ollama.generate()
Uses llama3.2:1b or llama3.2:3b
JSON Response
{"red_led": false,
"yellow_led": false,
"green_led": true}
parse_llm_response()
Parses JSON from SLM
Handles markdown code blocks
Returns (red, yellow, green) tuple
output_actuator()
Displays sensor data
Shows SLM JSON response
Shows SLM decision
Calls control_leds(red, yellow, green)
Shows final LED status

Autonomous system:

- Continuously monitored sensors
- Automatically decided LED states based on pre-defined rules
- No user interaction

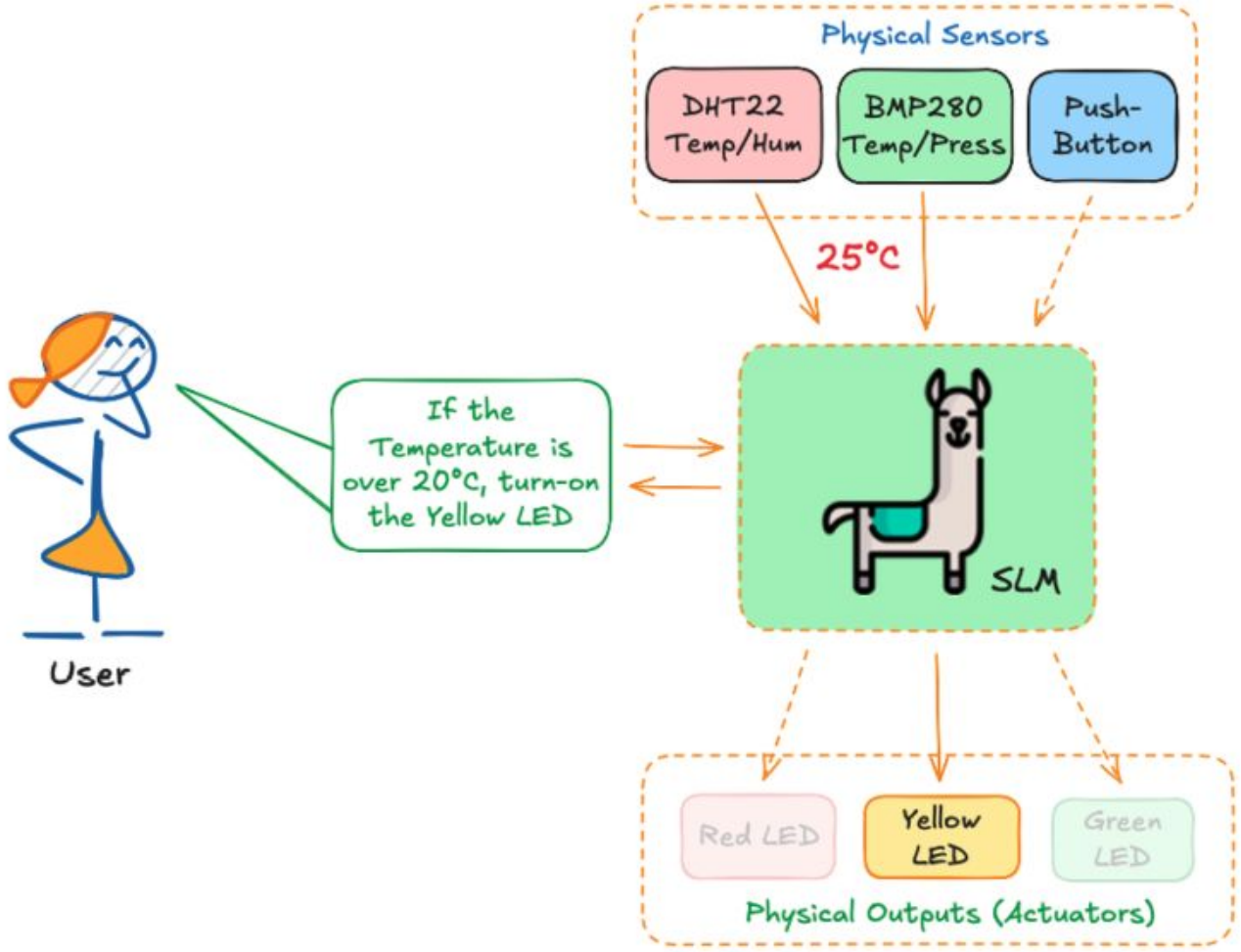


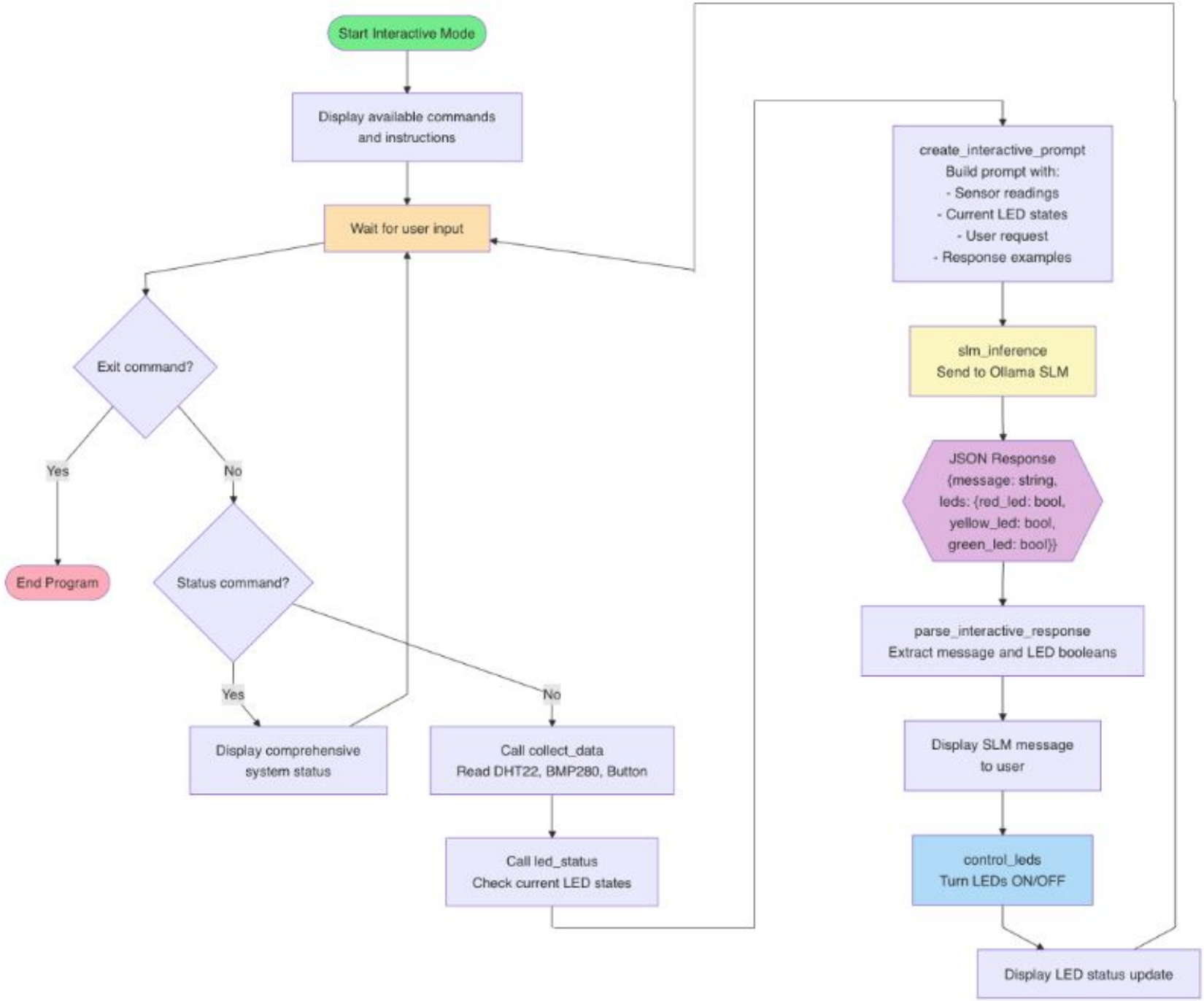
Interactions with Natural Language Commands

Interactive System:

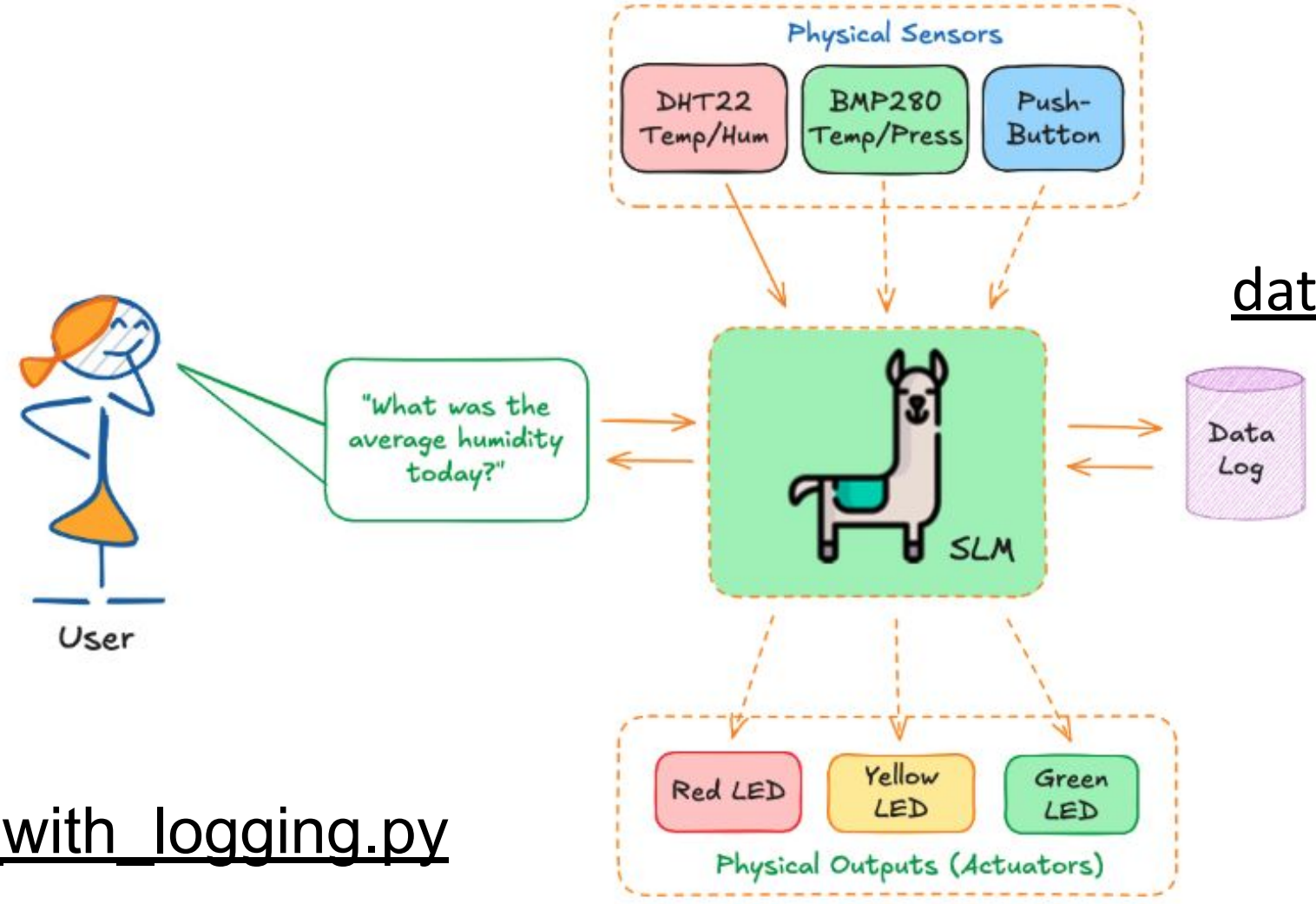
- Waits for user commands
- Accepts natural language queries and commands
- Provides conversational responses
- Executes actions based on user requests
- Displays comprehensive system status

slm_act_leds_interactive.py



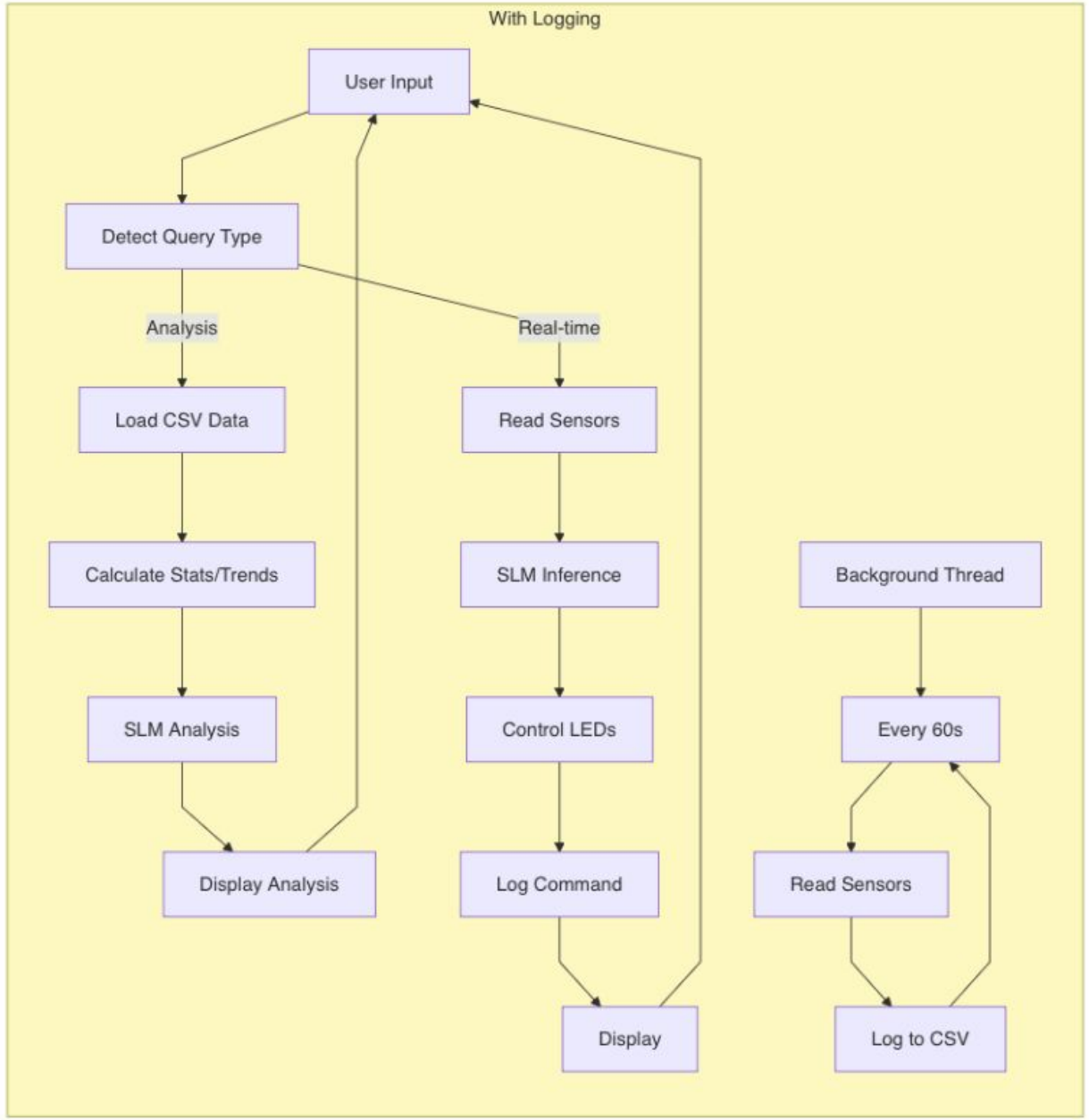

Start Interactive Mode
Display available commands and instructions
Wait for user input
Exit command?
Yes
No
End Program
Status command?
Yes
No
Display comprehensive system status
Call collect_data
Read DHT22, BMP280, Button
Call led_status
Check current LED states
create_interactive_prompt
Build prompt with:
- Sensor readings
- Current LED states
- User request
- Response examples
slm_inference
Send to Ollama SLM
JSON Response
{message: string,
leds: {red_led: bool,
yellow_led: bool,
green_led: bool}}
parse_interactive_response
Extract message and LED booleans
Display SLM message to user
control_leds
Turn LEDs ON/OFF
Display LED status update

Adding Data Logging



data_logger.py

slm_act_leds_with_logging.py


With Logging
User Input
Detect Query Type
Analysis
Real-time
Load CSV Data
Read Sensors
Calculate Stats/Trends
SLM Inference
Background Thread
SLM Analysis
Control LEDs
Every 60s
Display Analysis
Log Command
Read Sensors
Display
Log to CSV

Prompt Optimization and Efficiency

1. **System message**: It defines the assistant's behavior and should be sent once at initialization, not at PROMPT
2. **Prompt:** Should be drastically shortened
3. **API**: Ollama.Chat Instead of Ollama.Generate
4. **Model:** Pre-loading

```
Assistant: [Thinking...]
Assistant: All LEDs turned off.

LED Update: Red=OFF, Yellow=OFF, Green=OFF


Total Duration: 88.81 seconds
prompt_eval_duration: 80.41 s
load_duration: 1.98 s
eval_count: 32
eval_duration: 6.41 s
eval_rate: 4.99 tokens/s
```

```
Assistant: [Thinking...]
Assistant: All LEDs are now OFF

LED Update: Red=OFF, Yellow=OFF, Green=OFF


Total Duration: 13.01 seconds
prompt_eval_duration: 6.54 s
load_duration: 0.16 s
eval_count: 33
eval_duration: 6.31 s
eval_rate: 5.23 tokens/s
```

slm_act_leds_interactive_optimized.py

Using Pydantic

Using **Pydantic** is a robust way to improve the reliability, efficiency, and maintainability of our system, mainly since we rely on the LLM to output precise JSON

- **No more parsing errors** - Guaranteed valid JSON
- **Cleaner output** - No markdown code blocks
- **Type safety** - IDE autocomplete and type checking
- **Faster generation** - Constrained decoding is more efficient
- **Better validation** - Pydantic catches invalid values

slm_act_leds_interactive_pydantic.py

⚠️ IMPORTANT: The Notebook Kernel should end after using to liberate GPIOs

> The problem is that **Jupyter Notebook is still holding onto the GPIO pins** even after our code finishes running. When we try to run it from the terminal, those pins are already claimed by the Jupyter process. So, before running a code that deals with GPIO in the terminal, In Jupyter:
>
> - Click **Kernel → Restart Kernel**
> - Or: Click **Kernel → Shutdown Kernel**

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI